GH-600

Software Development Agent

Four-hour step-by-step workshop manual

Build a minimal Python agent workflow using GitHub, Copilot, OpenAI SDK, MCP, automated validation, pull requests, and human approval.

Workshop characteristic	Design
Duration	4 hours, including a 10-minute break
Repository	dev-agent-lab
Application complexity	Minimal Python; no web framework, database, or Docker
Control model	Agent proposes; CI validates; human approves; GitHub governs merge
Core outcome	Traceable issue-to-merge development lifecycle

Instructor-friendly • PowerShell commands • Complete file contents • Practical checkpoints

How to use this manual

Follow the stages in order. The final repository structure is shown first, but you should fill files only when each stage asks you to. Code examples are complete replacements for the named file unless explicitly marked as an additional configuration.

Workshop outcomes

• Create and test a small Python software development agent.

• Use an OpenAI model for structured planning without using Azure OpenAI.

• Expose a read-only task-status tool through MCP.

• Demonstrate state, audit events, specialist agent roles, and evaluation.

• Use GitHub Issues, feature branches, pull requests, GitHub Actions, required checks, and human approval.

• Explain GitHub security features and their availability/plan dependencies.

Safety and implementation boundaries

• Do not place API keys in source files, issue text, commits, logs, or screenshots.

• The LLM planner is separate from the deterministic application so local tests do not require API access.

• The MCP tool is read-only and does not run shell commands.

• The workshop does not automatically merge or deploy code.

• Repository rules and permissions—not prompts alone—enforce approval requirements.

Four-hour agenda

Stage	Topic	Time
1	Repository, specification, baseline app	20 min
2	Planning, prompts, OpenAI SDK	30 min
3	Tools and MCP	35 min
—	Break	10 min
4	State and audit trail	25 min
5	Multi-agent coordination	30 min
6	Evaluation and tuning	30 min
7	Issues, PRs, CI, security, human approval	50 min
8	End-to-end demo and recap	10 min
	TOTAL	240 min

GH-600 domain mapping

Domain	Where demonstrated
Architecture and SDLC	Stages 1–2: requirements, planning, workflow boundaries
Tools and environment	Stages 2–3: OpenAI, MCP, tool scope, errors
Memory and state	Stage 4: JSON state, audit log, recovery
Evaluation and tuning	Stage 6: tests, fixed inputs, failure analysis
Multi-agent coordination	Stage 5: planner, developer, reviewer, supervisor
Guardrails and accountability	Stage 7: PR checks, permissions, review, security

Final repository structure

```
dev-agent-lab/
■■■ app.py
■■■ llm.py
■■■ requirements.txt
■■■ .gitignore
■■■ tests/
■   ■■■ test_app.py
■■■ data/
■   ■■■ .gitkeep
■■■ docs/
■   ■■■ specs.md
■   ■■■ evaluation.md
■■■ prompts/
■   ■■■ plan.prompt.md
■■■ agents/
■   ■■■ planner.agent.md
■   ■■■ developer.agent.md
■   ■■■ reviewer.agent.md
■■■ skills/
■   ■■■ code-review/
■       ■■■ SKILL.md
■■■ mcp/
■   ■■■ server.py
■■■ .github/
    ■■■ copilot-instructions.md
    ■■■ instructions/
    ■   ■■■ tests.instructions.md
    ■■■ prompts/
    ■   ■■■ investigate.prompt.md
    ■■■ ISSUE_TEMPLATE/
    ■   ■■■ feature_request.yml
    ■■■ PULL_REQUEST_TEMPLATE.md
    ■■■ workflows/
    ■   ■■■ ci.yml
    ■■■ CODEOWNERS
    ■■■ dependabot.yml
    ■■■ SECURITY.md
```

Prerequisites

• Git and GitHub CLI (gh) installed and authenticated.

• Python 3.11 or later; CI uses Python 3.12.

• Visual Studio Code and GitHub Copilot access for Copilot-specific exercises.

• OpenAI developer API key and an available model name for the optional LLM exercise.

• A GitHub repository where you can configure Actions and security settings.

Check tools in PowerShell

```
git --version
gh --version
python --version
gh auth status
```

If the repository's default branch is not named main, use its actual branch name wherever this manual says main.

Stage 1 — Repository and baseline application (20 min)

Goal: establish the application boundary, repository instructions, first implementation, and unit tests.

Step 1.1 — Create repository and folders

```
mkdir dev-agent-lab
cd dev-agent-lab
git init
gh auth login
gh repo create dev-agent-lab --public --source . --remote origin
```

Use --private instead of --public if you want a private workshop repository.

```
$folders = @(
  "tests", "data", "docs", "prompts", "agents",
  "skills/code-review", "mcp",
  ".github/instructions", ".github/prompts",
  ".github/ISSUE_TEMPLATE", ".github/workflows"
)
foreach ($folder in $folders) {
  New-Item -ItemType Directory -Force $folder | Out-Null
}
```

Step 1.2 — Create docs/specs.md

```
# Software Development Agent

## Goal
Help a developer process a feature request through a controlled workflow.

## Input
A feature request describing a software change.

## Workflow
1. Validate the request.
2. Create a plan with acceptance criteria.
3. Propose an implementation.
4. Review the implementation.
5. Run automated validation.
6. Record decisions and results.
7. Require human approval before merging.

## Safety rules
- Never execute arbitrary shell commands from model output.
- Never hardcode or commit credentials.
- Never merge or deploy automatically in this workshop.
- Stop when a required validation check fails.
- Preserve evidence of the request and review.

## Success criteria
- Plans contain testable acceptance criteria.
- Changes are traceable to GitHub issues.
- Pull requests run automated checks.
- Reviewers can request changes.
- Only authorized humans approve changes.
```

Step 1.3 — Create .github/copilot-instructions.md

```
# Repository Instructions

- Read docs/specs.md before implementing features.
- Keep the application small and easy to understand.
- Use Python's standard library wherever practical.
- Keep OpenAI integration isolated in llm.py.
- Add or update tests for behavior changes.
- Do not hardcode API keys or credentials.
- Never execute arbitrary commands from model output.
- Never bypass failed validation or security checks.
- Do not merge or deploy changes automatically.
- Explain assumptions, files changed, and test results.
- Require human approval before consequential actions.
```

Step 1.4 — Replace app.py

```
import json
from pathlib import Path

ROOT = Path(__file__).parent
DATA_DIR = ROOT / "data"
STATE_FILE = DATA_DIR / "state.json"
AUDIT_FILE = DATA_DIR / "audit.jsonl"


def make_plan(request: str) -> dict:
    """Create a predictable plan for a feature request."""
    request = request.strip()
    if not request:
        raise ValueError("Feature request cannot be empty.")

    return {
        "request": request,
        "status": "planned",
        "steps": [
            "Understand the requirement",
            "Identify the smallest implementation change",
            "Add or update unit tests",
            "Run validation and request human review",
        ],
        "acceptance_criteria": [
            "The requested behavior is documented",
            "Tests cover expected behavior",
            "Validation completes successfully",
        ],
    }


def save_state(state: dict) -> None:
    """Save the latest workflow state."""
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    STATE_FILE.write_text(json.dumps(state, indent=2), encoding="utf-8")


def record_event(event: dict) -> None:
    """Append an event to the workshop audit log."""
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    with AUDIT_FILE.open("a", encoding="utf-8") as file:
        file.write(json.dumps(event) + "\n")


def main() -> None:
    request = input("Enter a feature request: ").strip()
    try:
        plan = make_plan(request)
    except ValueError as error:
        print(f"Error: {error}")
        return

    save_state(plan)
    record_event({
        "event": "plan_created",
        "request": request,
        "status": plan["status"],
    })
    print(json.dumps(plan, indent=2))
    print(f"\nState saved to: {STATE_FILE}")
    print(f"Audit log: {AUDIT_FILE}")


if __name__ == "__main__":
    main()
```

Step 1.5 — Replace tests/test_app.py

```
import unittest
from app import make_plan


class TestPlanner(unittest.TestCase):
    def test_creates_plan(self):
        plan = make_plan("Add task priority")
        self.assertEqual(plan["request"], "Add task priority")
        self.assertEqual(plan["status"], "planned")
        self.assertTrue(plan["steps"])
        self.assertTrue(plan["acceptance_criteria"])

    def test_rejects_empty_request(self):
        with self.assertRaises(ValueError):
            make_plan("")

    def test_rejects_whitespace_request(self):
        with self.assertRaises(ValueError):
            make_plan("   ")


if __name__ == "__main__":
    unittest.main()
```

Step 1.6 — Dependencies and .gitignore

```
requirements.txt:
openai
mcp

.gitignore:
__pycache__/
*.py[cod]
.venv/
.env
data/state.json
data/audit.jsonl
.pytest_cache/

python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
python app.py
```

Enter: Add a priority field to a task. Then inspect the state and audit files.

```
Get-Content .\data\state.json
Get-Content .\data\audit.jsonl

git add .
git commit -m "chore: initialize GH-600 development agent"
git push -u origin HEAD
```

Checkpoint: the baseline app runs, empty input is rejected, tests pass, and a plan is persisted.

Stage 2 — Planning, prompts, and OpenAI SDK (30 min)

Goal: compare predictable planning with LLM-assisted planning while keeping model output separate from execution.

Step 2.1 — Create prompts/plan.prompt.md

```
# Feature Planning Prompt

You are the planning agent for a software development team.

Given a feature request:
1. Restate the intended goal.
2. Break the work into small implementation steps.
3. Identify testable acceptance criteria.
4. Identify risks and missing information.
5. Do not claim to have changed code.
6. Request clarification if a requirement is ambiguous.

Keep the plan concise and suitable for human review.
```

Step 2.2 — Replace llm.py

```
import json
import os
from openai import OpenAI


def create_llm_plan(request: str) -> dict:
    """Ask an OpenAI model for a structured development plan."""
    if not request.strip():
        raise ValueError("Feature request cannot be empty.")
    if not os.getenv("OPENAI_API_KEY"):
        raise RuntimeError("Set OPENAI_API_KEY before using the LLM.")

    model = os.getenv("OPENAI_MODEL")
    if not model:
        raise RuntimeError("Set OPENAI_MODEL to an available model.")

    client = OpenAI()
    response = client.responses.create(
        model=model,
        instructions=(
            "You are a software planning agent. Return valid JSON with "
            "required fields: summary, steps, acceptance_criteria, risks. "
            "The last three fields must be arrays of strings. "
            "Do not claim code has been modified."
        ),
        input=request,
    )

    try:
        result = json.loads(response.output_text)
    except json.JSONDecodeError as error:
        raise ValueError("The model did not return valid JSON.") from error

    if not isinstance(result, dict):
        raise ValueError("The model response must be a JSON object.")

    required = {"summary", "steps", "acceptance_criteria", "risks"}
    if not required.issubset(result):
        raise ValueError("The model response is missing required fields.")
    if not isinstance(result["summary"], str):
        raise ValueError("summary must be a string.")

    for field in ("steps", "acceptance_criteria", "risks"):
        if not isinstance(result[field], list):
            raise ValueError(f"{field} must be a list.")
        if not all(isinstance(item, str) for item in result[field]):
            raise ValueError(f"Every item in {field} must be a string.")

    return result


if __name__ == "__main__":
    feature = input("Feature request: ")
    print(json.dumps(create_llm_plan(feature), indent=2))
```

Step 2.3 — Set OpenAI environment variables

```
$env:OPENAI_API_KEY = Read-Host "Enter your OpenAI API key"
$env:OPENAI_MODEL = Read-Host "Enter an available OpenAI model name"
python llm.py
```

Do not paste the key into source code, issue text, commits, or screenshots. This manual uses OpenAI's API, not Azure OpenAI.

Step 2.4 — Create .github/prompts/investigate.prompt.md

```
---
name: investigate
description: Investigate a feature request before implementation.
---

Read docs/specs.md and repository instructions.

For the supplied feature request:
1. Summarize the requirement.
2. Identify relevant files.
3. Propose the smallest implementation.
4. List acceptance criteria and tests.
5. Identify security risks and assumptions.

Do not modify files until the plan is reviewed.
Do not claim to have run tests unless you actually ran them.
```

In Copilot Chat, invoke /investigate if supported by your installed version. Compare its plan with the OpenAI planner's output.

Checkpoint: structured output is validated; the LLM proposes work but does not execute or approve it.

Stage 3 — Tools, MCP, and execution boundaries (35 min)

Goal: expose one read-only tool to an MCP-compatible client.

Step 3.1 — Replace mcp/server.py

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("dev-agent-tools")

MISSIONS = {
    "TASK-001": {
        "title": "Add task priority",
        "status": "planned",
        "owner": "developer-agent",
    },
    "TASK-002": {
        "title": "Add unit tests",
        "status": "in_review",
        "owner": "reviewer-agent",
    },
}


@mcp.tool()
def get_mission_status(mission_id: str) -> dict:
    """Return the status of a known development task."""
    mission = MISSIONS.get(mission_id)
    if mission is None:
        return {"found": False, "message": "Task not found"}
    return {"found": True, "mission": mission}


if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Step 3.2 — Configure VS Code MCP

```
New-Item -ItemType Directory -Force .vscode
```

```
.vscode/mcp.json:
{
  "servers": {
    "dev-agent-tools": {
      "type": "stdio",
      "command": "python",
      "args": ["mcp/server.py"]
    }
  }
}
```

MCP configuration varies by client and version. Use this schema only in a client that supports this VS Code-style configuration. Connect using a compatible MCP client and call get_mission_status with TASK-001 and an unknown ID.

Step 3.3 — Create skills/code-review/SKILL.md

```
---
name: code-review
description: Review changes against requirements, tests, and safety rules.
---

# Code Review Skill

1. Read docs/specs.md.
2. Identify the linked issue and acceptance criteria.
3. Inspect the complete diff.
4. Check input validation and error handling.
5. Check test coverage for valid and invalid inputs.
6. Look for exposed secrets and unsafe execution.
7. Report findings and return PASS or FAIL.
8. Never approve or merge a pull request automatically.
9. Never claim tests passed without validation evidence.
```

Step 3.4 — Discuss tool failures

Scenario	Expected handling
Unknown task	Return a clear not-found result.
Server unavailable	Stop or report actionable error.
Read fails	Retry only if safe and appropriate.
Validation fails	Do not advance to approval or merge.
Extra privilege needed	Request an authorized decision.

Checkpoint: the MCP tool is read-only. MCP standardizes tool discovery and calls; it does not automatically make tools safe.

Stage 4 — Persistent state and audit trail (25 min)

Goal: understand how a task resumes with its prior decisions and how events are recorded.

Step 4.1 — Create .github/instructions/tests.instructions.md

```
---
applyTo: "tests/**/*.py"
---

# Test Instructions

- Use Python's unittest framework.
- Test expected behavior and invalid input.
- Do not call external services in unit tests.
- Keep tests deterministic.
- Never use real API keys in test fixtures.
- Run: python -m unittest discover -s tests -v
- Report the actual test result.
```

Step 4.2 — Inspect the state and event log

```
python app.py
Get-Content .\data\state.json
Get-Content .\data\audit.jsonl
```

Run app.py a second time with another request. state.json stores the latest plan; audit.jsonl appends events from prior runs.

Step 4.3 — Discuss memory types

Type	Example	Purpose
Short-term context	Current request and plan	Supports current reasoning
Durable task state	data/state.json	Resumes work across runs
Audit history	data/audit.jsonl	Retains events
External records	GitHub issue/PR	Collaboration and review history

Step 4.4 — Recovery discussion

Ask Copilot: If this agent resumes tomorrow, what must it re-read and validate? How should it handle stored state that conflicts with docs/specs.md?

Expected answer: reload the request, approved plan, current status, completed work, outstanding tasks, and latest validation. Revalidate state rather than trusting old model output.

Note: the JSONL log is not a tamper-proof audit system. Production audit needs identity, timestamps, access control, retention, and integrity protection.

Stage 5 — Multi-agent orchestration (30 min)

Goal: coordinate three specialist roles through a supervisor, without adding an agent framework.

Step 5.1 — agents/planner.agent.md

```
---
name: planner
description: Creates small implementation plans from feature requests.
---

Read docs/specs.md and the request.
Return the goal, implementation steps, testable acceptance criteria,
risks, and missing information.
Do not modify files or claim implementation is complete.
Wait for human approval of the plan before implementation.
```

Step 5.2 — agents/developer.agent.md

```
---
name: developer
description: Proposes minimal, testable code changes.
---

Read the approved plan and repository instructions.
Implement the smallest change satisfying acceptance criteria.
Add or update tests.
Explain assumptions and files changed.
Run validation and report actual results.
Do not bypass checks, merge, or deploy automatically.
```

Step 5.3 — agents/reviewer.agent.md

```
---
name: reviewer
description: Reviews changes against requirements and safety rules.
---

Inspect the complete diff and linked issue.
Evaluate acceptance criteria, correctness, input validation,
test coverage, security risks, validation evidence, and assumptions.
Return PASS or FAIL with findings.
Do not modify the implementation or approve your own changes.
A human reviewer makes the final approval decision.
```

Step 5.4 — Run the handoff

- Ask planner to plan the task-priority issue.
- Human reviews and approves the plan.
- Ask developer to implement the plan.
- Run unit tests and inspect the diff.
- Ask reviewer to evaluate the implementation.
- If review fails, send findings back to developer.
- Repeat validation after corrections.
- Submit the PR for human approval.

Supervisor flow

```
Feature request
  -> Supervisor
  -> Planner
  -> Human plan approval
  -> Developer
  -> Automated tests
  -> Reviewer
  -> Human PR approval
  -> GitHub-controlled merge
```

These Markdown profiles describe roles; they do not automatically orchestrate themselves. The supervisor is the controlled sequence and its decision gates. GitHub permissions enforce the repository boundary.

Checkpoint: roles have clear boundaries, handoffs, failure handling, and human approval.

Stage 6 — Evaluation and tuning (30 min)

Goal: measure agent output against repeatable criteria instead of judging whether it sounds convincing.

Step 6.1 — Replace docs/evaluation.md

```
# Agent Evaluation

Evaluate each proposed plan and implementation.

| Criterion | Passing condition |
|---|---|
| Completeness | Goal, steps, and acceptance criteria are present |
| Relevance | Steps address the actual request |
| Testability | Criteria can be verified |
| Safety | No unauthorized execution is proposed |
| Traceability | Request and decision are recorded |
| Validation | Required automated checks pass |

## Failure handling
1. Record the failed criterion.
2. Identify the cause.
3. Correct the relevant prompt, context, tool, or code.
4. Rerun the same test case.
5. Confirm previously passing tests still pass.

## LLM evaluation
Use fixed requests for repeatable comparisons.
Validate JSON and required fields before scoring content.
Record model, prompt version, inputs, and results.
Do not rely on a single successful response.
```

Step 6.2 — Run baseline checks

```
python -m unittest discover -s tests -v
python -m compileall -q app.py llm.py mcp tests
```

Step 6.3 — Evaluate three requests

```
Add a priority field to a task.
Add a priority field and tests for valid and invalid values.
Make the application secure.
```

Compare completeness, relevance, testability, and safety. The vague request should trigger clarification rather than invented requirements.

Know the difference

Technique	Question answered
Unit test	Does this code behave as expected?
Agent evaluation	Does the output meet quality and safety criteria?
Regression test	Did a change break previously passing behavior?
Tuning	Did changing prompts, context, tools, or code improve results?

Checkpoint: use repeatable inputs, record failure causes, tune the relevant component, and rerun evaluation.

Stage 7 — GitHub SDLC, validation, and human approval (50 min)

Goal: make the agent work inside the normal GitHub development lifecycle. The agent proposes; CI validates; a human approves; GitHub controls merge.

Step 7.1 — Create .github/ISSUE_TEMPLATE/feature_request.yml

```
name: Feature request
description: Propose a small, testable feature.
title: "[Feature]: "
labels:
  - enhancement
body:
  - type: textarea
    id: problem
    attributes:
      label: Problem statement
      description: What problem needs to be solved?
      placeholder: Developers need to prioritize tasks.
    validations:
      required: true
  - type: textarea
    id: expected
    attributes:
      label: Expected behavior
      description: What should the application do?
    validations:
      required: true
  - type: textarea
    id: acceptance
    attributes:
      label: Acceptance criteria
      description: List testable outcomes.
    validations:
      required: true
  - type: dropdown
    id: risk
    attributes:
      label: Change risk
      options:
        - Low
        - Medium
        - High
    validations:
      required: true
  - type: checkboxes
    id: confirmations
    attributes:
      label: Confirmations
      options:
        - label: I have not included secrets or sensitive data.
          required: true
        - label: The expected behavior can be tested.
          required: true
```

Step 7.2 — Create a feature issue

• Commit the issue template to the default branch.

• Open Issues > New issue and select Feature request.

• Title: [Feature]: Add task priority.

• Problem: Developers cannot distinguish urgent tasks from normal tasks.

• Expected behavior: each task has Low, Medium, or High priority.

• Acceptance: valid values are accepted, invalid values rejected, tests cover both.

• Set risk to Low, submit, and record the issue number (example #1).

Step 7.3 — Create a feature branch

```
git switch main
git pull
git switch -c feature/issue-1-task-priority
```

Replace main and issue-1 with your actual default branch and issue number. Ask Copilot to investigate the issue, read docs/specs.md, propose a minimal plan, and list tests before changing files.

Step 7.4 — Create .github/PULL_REQUEST_TEMPLATE.md

````
## Summary

Describe the change and the problem it solves.

## Related issue

Closes #

## Implementation

- [ ] Follows repository instructions.
- [ ] Changes only necessary files.
- [ ] Does not introduce secrets or credentials.
- [ ] Does not introduce unauthorized execution.

## Acceptance criteria

- [ ] Criterion 1 is satisfied.
- [ ] Criterion 2 is satisfied.
- [ ] Criterion 3 is satisfied.

## Validation evidence

- [ ] Unit tests passed.
- [ ] GitHub Actions checks passed.
- [ ] Relevant security findings were reviewed.
- [ ] Manual validation was completed where necessary.

Commands executed and actual results:

```text
Record commands and outcomes here.
```

## Human review

- [ ] I inspected the complete diff.
- [ ] I checked acceptance criteria.
- [ ] I reviewed security and failure handling.
- [ ] Remaining risks are documented.

## Remaining risks

List limitations, assumptions, and follow-up work.
````

Stage 7 — Continue: CI, rules, security, and review

Step 7.5 — Create .github/workflows/ci.yml

```
name: CI

on:
  push:
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

jobs:
  test:
    name: test
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Check Python syntax
        run: python -m compileall -q app.py llm.py mcp tests

      - name: Run unit tests
        run: python -m unittest discover -s tests -v
```

Commit this workflow to the default branch before making its status check required. In production, use supported action versions and consider pinning third-party actions to reviewed commit SHAs.

Step 7.6 — Run local checks and create the PR

```
python -m unittest discover -s tests -v
python -m compileall -q app.py llm.py mcp tests
git status
git diff --check
git diff

git add .
git commit -m "feat: add task priority for issue #1"
git push -u origin feature/issue-1-task-priority
gh pr create --base main --head feature/issue-1-task-priority --title "Add task priority" --body "Closes #1. Adds task priorit
```

The code must already be implemented and tested before these commit/PR commands. The PR template is applied if it exists on the default branch when the PR is created.

Step 7.7 — Inspect Actions checks

• Open Actions and select the latest CI run.

• Open the test job and inspect syntax and test output.

• Return to the PR and inspect the check result.

• Demonstrate a failure on a disposable branch by adding self.assertEqual(1, 2) to a test.

• Commit and push the deliberate failure; inspect the failed log.

• Remove the assertion, rerun tests, commit the correction, and push.

• Confirm a new run starts and passes.

Step 7.8 — Require checks and approval

• Open Settings > Rules > Rulesets.

• Create a ruleset targeting the default branch.

• Require a pull request before merging.

• Require at least one approval.

• Require the test status check; run CI first so the check is available.

• Where appropriate, dismiss stale approvals when the target branch changes.

• Block force pushes and branch deletion; save and test the ruleset.

Rules and bypass enforcement vary by plan and repository permissions. Do not give the agent a bypass just to simplify the workshop.

Step 7.9 — CODEOWNERS

```
.github/CODEOWNERS:
* @YOUR-GITHUB-USERNAME
```

Replace the placeholder with an actual GitHub username or team. CODEOWNERS identifies responsible reviewers; a ruleset must enforce code-owner approval.

Stage 7 — Security and human review

Step 7.10 — Dependabot

```
.github/dependabot.yml:
version: 2

updates:
  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "weekly"

  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
```

Dependabot needs a supported dependency manifest. requirements.txt supplies the Python packages for this project.

Step 7.11 — Security policy

```
.github/SECURITY.md:
# Security Policy

## Scope
This repository is a workshop demonstration, not a production service.

## Reporting vulnerabilities
Do not publish credentials, customer data, or sensitive vulnerability
details in a public issue. Use the repository owner's approved private
reporting channel.

## Workshop security rules
- Never commit API keys or credentials.
- Revoke exposed credentials; deleting them from the latest commit
  does not guarantee removal from Git history.
- Review dependency and code-scanning findings.
- Do not grant the agent unnecessary permissions.
- Do not merge changes that fail required checks.
```

Step 7.12 — Demonstrate available security features

Capability	Workshop demonstration
Secret scanning	Explain detection, credential revocation, and prevention.
Dependabot	Inspect dependency alerts or an update PR.
CodeQL	Enable supported Python code scanning and inspect findings.
Dependency graph / SBOM	Inspect dependency data and demonstrate a supported SBOM export.
Security policy	Show vulnerability-reporting guidance.
Rulesets	Demonstrate required check and review enforcement.

Availability depends on repository visibility, licensing, organization settings, and permissions. Dependabot does not automatically produce an SBOM, and a security alert does not automatically become a required PR status check.

Step 7.13 — Human review and decision

• Inspect the complete diff and confirm acceptance criteria.

• Check tests cover expected and invalid inputs.

• Verify the latest CI result passed.

• Review relevant security findings and unresolved comments.

• Request changes if needed; return to developer and rerun validation.

• Approve only when satisfied; merge using GitHub's permitted interface.

The PR body should include Closes #1. GitHub closes the linked issue when the PR is merged into the default branch.

Key distinction: a PR template is guidance; required status checks and branch rules enforce policy. A passing CI run does not replace human review.

Stage 8 — End-to-end demonstration and recap (10 min)

Goal: demonstrate the entire lifecycle from issue intake to controlled merge.

Run the scenario

• Submit a feature request through the GitHub issue template.

• Ask planner to produce a plan; review and approve it.

• Ask developer to implement on a feature branch.

• Run unit tests locally and inspect the diff.

• Open a pull request using the PR template and link the issue.

• Observe GitHub Actions validation.

• Ask reviewer to inspect the change; request changes once for demonstration.

• Fix the issue, rerun checks, and obtain human approval.

• Merge through GitHub after required checks and approval pass.

• Verify issue closure, merged PR, checks, and audit files.

Useful PowerShell and GitHub CLI commands

```
Get-Content .\data\state.json
Get-Content .\data\audit.jsonl

gh issue view 1
gh pr view --web
gh pr checks
gh run list --limit 5
gh run view RUN_ID --log-failed
gh pr list --state merged
gh issue list --state closed
```

Replace example issue numbers and RUN_ID with actual values.

Workflow state labels for discussion

State	Meaning
planned	A plan exists
in_progress	Work is being performed
validation_failed	A required check failed
changes_requested	Human review requests corrections
ready_for_review	Validation passed; awaiting review
approved	Authorized reviewer approved the current proposal
merged	GitHub confirms the PR was merged

These are workshop-level labels, not all built-in GitHub statuses. GitHub's actual check and review states remain authoritative.

Final instructor checklist

• ■ Application and unit tests work without a web framework.

• ■ OpenAI planner returns validated structured output.

• ■ MCP status tool works and handles unknown IDs.

• ■ State and audit files persist across runs.

- ■ Planner, developer, reviewer, and supervisor roles are explained.
- ■ A failing CI run and subsequent passing run were demonstrated.
- ■ Issue and PR templates are available on the default branch.
- ■ Required checks and review rules are configured or limitations explained.
- ■ Human review, merge, and linked issue closure were demonstrated.
- ■ Security capabilities and plan dependencies were discussed.

Troubleshooting and instructor notes

Common issues

Issue	What to check
gh command not found	Install GitHub CLI, reopen PowerShell, run gh auth login.
Python package import fails	Activate .venv and run python -m pip install -r requirements.txt.
OpenAI authentication error	Check OPENAI_API_KEY in the current session and model availability.
LLM response is not JSON	Review the prompt and keep schema validation; do not trust malformed output.
MCP client cannot connect	Check client-specific config schema, Python path, package install, and server logs.
Actions does not run	Check workflow path, YAML syntax, Actions permissions, and branch.
Required check unavailable	Run the workflow first and select the correct job/status name.
Issue does not close	Check the PR body uses Closes #NUMBER and was merged into the default branch.
Security setting unavailable	Check repository visibility, plan, organization policy, and permissions.

Official references

• GH-600 study guide: https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/gh-600

• GH-600 course: https://learn.microsoft.com/en-us/training/courses/gh-600t00

• GitHub Copilot docs: https://docs.github.com/en/copilot

• GitHub Actions docs: https://docs.github.com/en/actions

• GitHub repository rules: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository

• GitHub security overview: https://docs.github.com/en/code-security

• GitHub Agentic Workflows: https://github.github.com/gh-aw/

• OpenAI API docs: https://platform.openai.com/docs/

• Model Context Protocol: https://modelcontextprotocol.io/

Final principle

The agent may plan and propose changes. Automated checks supply validation evidence. Authorized humans retain approval authority. GitHub permissions and repository rules—not a prompt alone—control whether changes can merge.